

Technology Stack

- **Mobile App:** React Native (iOS & Android native), Reanimated 3, MapLibre GL radar
- **Backend / API:** Python 3.11 (**FastAPI** Async), Node.js Gateway, WebSockets (**WSS**), REST
- **AI / ML & Graph DL:** PyTorch Geometric (**ST-GAT** Corridor Attention), ONNX Runtime (<10ms)
- **Physics Kinematics:** Newton-Davis dynamic tractive engine, grade, air drag & RK4 integrator
- **Data Streaming:** Apache Kafka (100k+ event/s KRaft), Redis 7.2 (**Pub/Sub** in-memory cache)
- **Spatial & Time-Series:** PostgreSQL 16 + **PostGIS** (1D rail chainage LRS), TimescaleDB
- **Railway Ingestion:** BEL RTIS (NavIC 30s GPS), CRIS FOIS freight, COA logs, e-Caution TSR
- **Security & Hosting:** OAuth 2.0 / JWT, AES-256 GCM, Docker, Kubernetes on RailCloud / NIC

100% ZERO-HARDWARE Taps existing statutory Indian Railways GPS & CRIS feeds. No engine modifications.

Powered By



Python



PyTorch



React Native



Kafka



Redis



PostgreSQL



PostGIS



FastAPI



Docker



Kubernetes



ISRO NavIC



CRIS / IR

⚡ GATI-SETU: End-to-End Operational Decision Architecture

Closed-Loop Ground Reality Engine

30s NavIC Satellite Ingestion • Newton-Davis Kinematics + ST-GAT AI • 3 Primary User Action Outcomes

1 Ingestion Stage
100% Zero-Hardware Telemetry**User / Station Ingress**

Passenger enters Train #12301 or GPS auto-detects boarding point

**BEL RTIS NavIC Feed**

30s GPS coordinates & velocity streamed via ISRO GSAT-15

**RTIS GPS Fix Valid?**

Satellite Handshake Check

NO / Shadow

**Dead-Reckoning**

Newton-Davis kinematics in tunnels/gorges

YES / Fix

**1D Track-Snap**

PostGIS LRS snaps to track chainage KM

**Apache Kafka & Redis Bus**

100k+ events/sec distributed telemetry ingest across 14,000+ trains

**2 Processing Stage**
Physics Kinematics & AI Core**Dynamic Kinematics Solver**

$$a(t) = [F_{\text{trac}}(v) - R_{\text{davis}}(v) - F_{\text{grade}}] / M_{\text{eff}}$$

WAP-7 tractive effort, trailing tonnage inertia & gradient resistance

**ST-GAT AI & Conformal Engine**

$$a_{ij} = \text{Softmax}(\text{LeakyReLU}(a^T [Wh_i || Wh_j]))$$

Predicts cascading network delays & computes calibrated **[P10, P50, P90]** ETA**Delay > 5 Mins Detected?**

Schedule Deviation & Precedence Gate

↗ [NO: ≤ 5m] Nominal

↘ [YES: > 5m] Mitigation

**Real-Time Operational Dispatcher**

Sub-10ms WebSocket & gRPC broadcast to all 3 stakeholder cockpits

3 Output Delivery Stage
3 Mandated Primary User Cockpits**NOMINAL FLOW (Deviation ≤ 5 Mins)****Passenger Mobile App**

Live [P10-P90] arrival window, station radar & platform walk guide

**Section Controller Cockpit**

130 km/h MPS clear corridor; automatic 4-aspect green wave headway

**Station Master & PIDS**

Direct route relay lock confirmed & multilingual audio/LED sync

DELAY MITIGATION (Deviation > 5 Mins)**Passenger: Root-Cause & City Sync**

"Held at Outer Signal for PF-4" + 1-tap auto Metro/Cab re-booking

**Controller: Rule 401 Siding Advisory**

Loops slow freight to siding at KM 142 to save 22m coaching delay

**Station: Outer Hold & Berth Shift**Predicts ΔT_{outer} & dynamically shifts rake to PF-2 to clear outer hold